

Multi-Model Intrusion Detection on UNSW-NB15

Methodology, Results and Analysis — a pedagogical walkthrough

This document explains every stage of the project from first principles: what each transformation does, why it is done, what the alternatives were, what the result means, and where its limits lie. It is written so that you can fully own and defend the work. All numbers are taken directly from the project's run artifacts; figures are the exact ones produced by the analysis scripts.

Project: CIS*6560 Research Project, MCTI, University of Guelph. **Dataset:** UNSW-NB15 (partitioned set).

Protocol: 5 random seeds, identical pipeline across all model families.

Contents

1. The big picture
2. Data integrity and ingestion
3. Exploratory data analysis (EDA)
4. Preprocessing pipeline
5. Class-imbalance handling
6. Feature engineering
7. Models and architectures
8. Evaluation protocol and headline results
9. RQ1 — feature-engineering strategy and feature groups
10. RQ2 — which classes stay hard, and why
11. Limitations and honest scope
12. Explain this project in three minutes
13. Glossary

1. The big picture

An **intrusion detection system (IDS)** watches network traffic and flags activity that looks like an attack. This project does not chase a single best detector; its contribution is a **fair, like-for-like comparison** of three families of models under one identical pipeline: simple *rule-based* logic, *classical machine learning* (Random Forest, SVM, XGBoost), and *deep learning* (a 1-D CNN and an LSTM). Every model sees exactly the same data splits, preprocessing, and metrics, so any difference in results is attributable to the model, not to inconsistent treatment.

Two research questions drive the analysis:

- **RQ1:** Which feature-engineering strategy, and which kinds of features, best support detection of each attack type?
- **RQ2:** Which attack classes remain hard to detect regardless of model, and *why* (false positives and class confusion)?

The data flows through eight stages, shown in the map below: an integrity check, exploratory analysis, preprocessing, imbalance handling, feature engineering, model training, evaluation, and finally the visual analysis that answers the two questions. The rest of this document walks each stage in that order.

UNSW-NB15 — Data Transformation Mind Map

From integrity check to publication figures · multi-model IDS · 5 seeds (42-46) · 10 attack classes

1 · INTEGRITY & INGEST

SHA-256 checksum verify

Confirm download intact, untampered

Load partitioned set

257,673 rows × 45 columns

Pool train + test

One corpus for a unified split

Schema / range validation

Reject bad dtypes & out-of-range

2 · EXPLORATORY ANALYSIS (9 methods → 8 figures)

Class distribution

Reveals severe 10-class imbalance

Correlation & skewness

Find redundant, heavy-tailed feats

PCA & t-SNE

Inspect class separability

Outliers · ANOVA · tests

Rank discriminative features

3 · PREPROCESSING (fit on TRAIN only — no leakage)

Drop id, deduplicate

94,928 dupes (36.8%) removed

Infinites + median impute

Clean artifacts; train-fit medians

One-hot proto/service/state

→ 190 features; rare bucketed

Stratified 70/15/15 split

Keep class ratios across folds

Z-score standardize

Scaled for NN, raw for trees

Sliding windows (W=8)

Build sequences for the LSTM

4 · IMBALANCE HANDLING (train fold only)

Inverse-freq class weights

Penalize rare-class errors

SMOTE / Borderline variants

Synthesize ultra-rare samples

Stratified subsample

RBF-SVM 30k cap ($O(n^3)$ cost)

5 · FEATURE ENGINEERING

Filter: MI + ExtraTrees

Consensus rank of two filters

RFA (RF & SVM variants)

Greedy add; 14 feats win

Flow-pair (bigram) feats

Pair consecutive flow records

Ablation under fixed RF

Compare strategies fairly

6 · MODELS & ARCHITECTURES (identical pipeline, 5 seeds)

Rule-based

Depth-5 tree → readable rules

Random Forest

Class-balanced, scale-invariant

SVM (RBF)

30k subsample, internal scaler

XGBoost

Boosted trees, native NA handling

1D-CNN

Conv1D-BN-ReLU-Pool → GAP → Dense

LSTM

Window 8 → LSTM → softmax

DL input + training recipe

signed-log1p + z-score · lr 5e-4 · dropout 0.3 · ReduceLROnPlateau · early stopping

7 · EVALUATION (val + test, 5 seeds, bootstrap CIs)

Per-class metrics

Macro-F1, recall, one-vs-rest FPR

Binary curves

AUROC and PR-AUC

Bootstrap 95% CI

Quantify seed-to-seed variance

Confusion + train/test

Expose the generalization gap

8 · VISUALIZATION & RESEARCH ANSWERS

DL train/val curves

Prove the deep models don't overfit

Train-vs-test gap bars

Generalization by model family

RQ2 per-class F1 heatmap

Hard classes across all models

RQ2 difficulty ranking

Hardest→easiest vs support

RQ1 strategy comparison

RFA-14 beats the 190-feat baseline

RQ1 RFA group makeup

basic_flow = 10 of 14 features

RQ1 feature-group importance heatmap

Permutation: which group drives each attack class

Figure 1. The full data journey, stage by stage, with a one-line rationale per step.

Why this matters. A comparison is only meaningful if it is fair. Fixing one pipeline and varying only the model is what lets us make defensible claims like "trees beat deep nets here" without the result being an artifact of uneven preprocessing.

Check your understanding. In your own words, why is it important that all six models share the same train/validation/test split and the same metrics?

2. Data integrity and ingestion

What it is. Before any analysis, each downloaded data file is checked against a stored **SHA-256 checksum** — a 64-character fingerprint computed from the file's bytes. If a single byte changed (corrupted download, wrong version, tampering), the fingerprint would not match and the pipeline stops.

Why it matters. Every downstream number depends on the exact input bytes. A silent corruption would invalidate the entire study, and you would not know. The checksum makes the starting point reproducible and verifiable by anyone who re-runs the project.

What we load. The project uses the UNSW-NB15 *partitioned* distribution: a training partition of 175,341 records and a test partition of 82,332, pooled to 257,673 records with 45 columns. We pool them so we can apply our own controlled, stratified split rather than inherit the dataset authors' split. A lightweight **schema and range validation** then rejects wrong data types and impossible values (for example a negative byte count) before they can poison the statistics.

Check your understanding. If two people download the dataset on different days and get the same SHA-256 value, what does that guarantee, and what does it not guarantee?

3. Exploratory data analysis (EDA)

What it is. EDA is the "get to know your data" phase: nine analyses that profile the class balance, the feature distributions, the correlations between features, and how separable the attack classes are. It produces no model; it produces understanding that shapes every later decision.

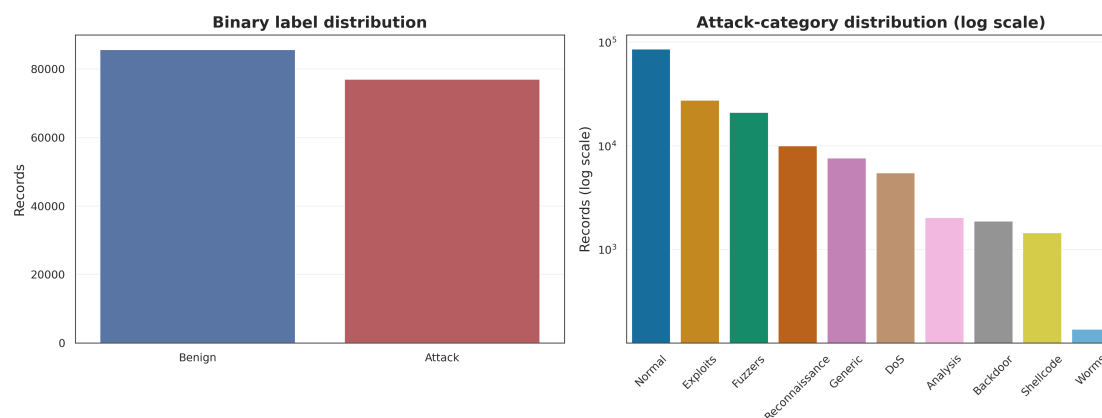


Figure 2. Class distribution. The dataset is dominated by Normal traffic and a few large attack types; several attack classes are extremely rare.

The single most important finding is **severe class imbalance**. In the test split, Normal traffic has 12,859 records while Worms has only 25, and Analysis, Backdoor and Shellcode each have only a few hundred. This imbalance is the central difficulty of the whole project: a model can score high "accuracy" simply by ignoring the rare classes, so we must choose metrics and training tricks that refuse to let it cheat (Sections 5 and 8).

Correlation and skewness analyses reveal two more things that directly drive preprocessing: many features are **heavily right-skewed** with enormous values (byte counts and load rates reach into the billions), and some features are redundant. The PCA and t-SNE projections (below) compress the 40-plus features into two dimensions so we can see, by eye, that some attack types form tight separable clusters while others

overlap heavily with Normal traffic — an early warning of which classes will be hard.

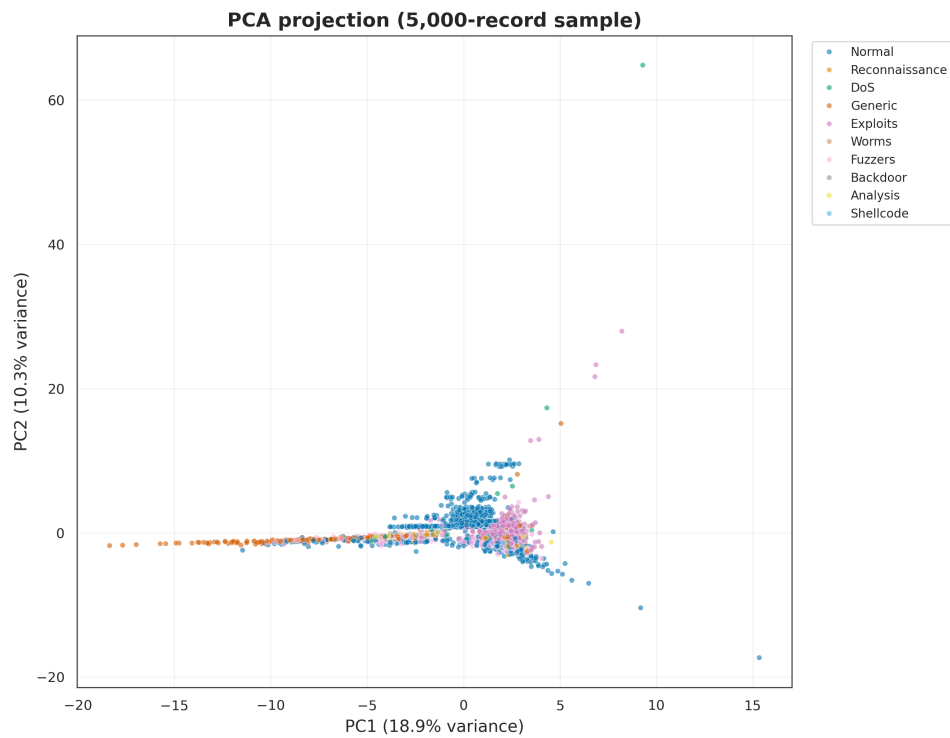


Figure 3. PCA projection. Linear structure: some classes separate, others overlap, hinting at intrinsic difficulty.

Why this matters. EDA is where you earn the right to make modeling choices. The imbalance finding justifies class weights and macro-F1; the skew finding justifies the log transform for neural nets; the overlap seen in PCA/t-SNE foreshadows the RQ2 "hard classes" result.

Check your understanding. Why can a classifier reach 90%+ raw accuracy on this dataset while being almost useless at catching Worms?

4. Preprocessing pipeline

Preprocessing turns raw, messy records into clean numerical matrices, while obeying one rule above all: **no leakage**. Anything learned from the data (medians, scaling statistics, encodings) is computed on the **training fold only** and then applied to validation and test. If we fit those on the full dataset, the model would indirectly "see" the test set and our scores would be optimistically wrong.

Cleaning. We drop the artificial *id* column (an index, not a signal), remove duplicate records, and neutralize infinities. Deduplication is significant here: 94,928 records (36.8% of the pool) are exact duplicates. Leaving them in would let the model memorize repeated rows and would inflate scores. Missing values are filled with the **median** of each feature computed on training data only (the median is robust to the heavy skew we found in EDA).

Encoding. Three columns are categorical text (protocol, service, connection state). Models need numbers, so we **one-hot encode** them: each category becomes its own 0/1 column. Rare categories are bucketed together to avoid a blow-up of near-empty columns. After encoding, the feature count grows to 190.

Splitting and scaling. We make a **70/15/15 stratified split** into train, validation and test, stratified on the attack label so every class appears in the same proportion in each fold. Scaling uses a **dual treatment**, and understanding why is important: tree models (Random Forest, XGBoost, the rule tree) are *scale-invariant* — they split on thresholds, so multiplying a feature by a million changes nothing — and they are fed the raw, unscaled features. Neural networks are the opposite: they do gradient descent and require inputs on a comparable, modest scale. They receive **z-scored** features (each feature shifted and rescaled to mean 0, standard deviation 1). A separate **sliding-window** builder stacks consecutive records into short sequences for the LSTM.

Why this matters. The dual scaling is not an accident; it is the difference between a working deep model and a broken one. Section 7 shows that feeding raw billion-scale features to the networks collapses them, and that fixing the scaling is what rescued them.

Check your understanding. We compute the median for imputation on the training fold only, then reuse it on the test fold. What specific error would we be committing if we instead computed it over all data?

5. Class-imbalance handling

Because rare classes are easy to ignore, we counter the imbalance in three complementary ways, all applied to the **training fold only** (resampling the test set would fake the results):

- **Inverse-frequency class weights:** the loss function is told that a mistake on a rare class costs more than a mistake on a common one, in proportion to how rare the class is. This is the lightest-touch fix and is used by every model family.
- **SMOTE and its variants:** synthesize new, plausible minority-class examples by interpolating between real ones, rather than just copying. This gives the model more rare-class signal to learn from. (Borderline variants were noted to leave the ultra-rare Worms class almost untouched, a documented limitation.)
- **Stratified subsampling:** used for the RBF-SVM, whose training cost grows roughly with the square of the number of records; we cap it at 30,000 stratified rows so it remains tractable.

Check your understanding. Why must SMOTE be applied after the train/test split and only to the training fold?

6. Feature engineering

Feature engineering asks: do we need all 190 features, and can a cleverly chosen subset do better? Four strategies are compared, all judged under one fixed Random Forest so the comparison isolates the *features*, not the model:

- **Filter (consensus):** rank features two ways — by mutual information (how much a feature tells us about the label) and by Extra-Trees importance — and keep the features both methods agree on. Fast, model-light.
- **Recursive Feature Addition (RFA):** start from nothing and greedily add the feature that most improves validation score, stopping when it stops helping. Two variants are implemented: the original SVM cost-function form (Hamed, Dara and Kremer 2018) and a Random-Forest / validation-F1 form.
- **Flow-pair (bigram) features:** pair each record with its predecessor to capture short-range structure. This is cited honestly as *inspired by* the original payload-bigram technique, not identical to it (the partitioned dataset has no payloads).
- **Baseline:** all 190 features, no selection.

The ablation result is decisive and slightly counter-intuitive: **fewer, better-chosen features win**. The 14-feature RFA selection scores the highest multiclass macro-F1 (0.631), beating the full 190-feature baseline (0.530) and even the 394-feature flow-pair set (0.504), which is the *worst*. More features added noise and redundancy; targeted selection added signal and was also far cheaper to train.

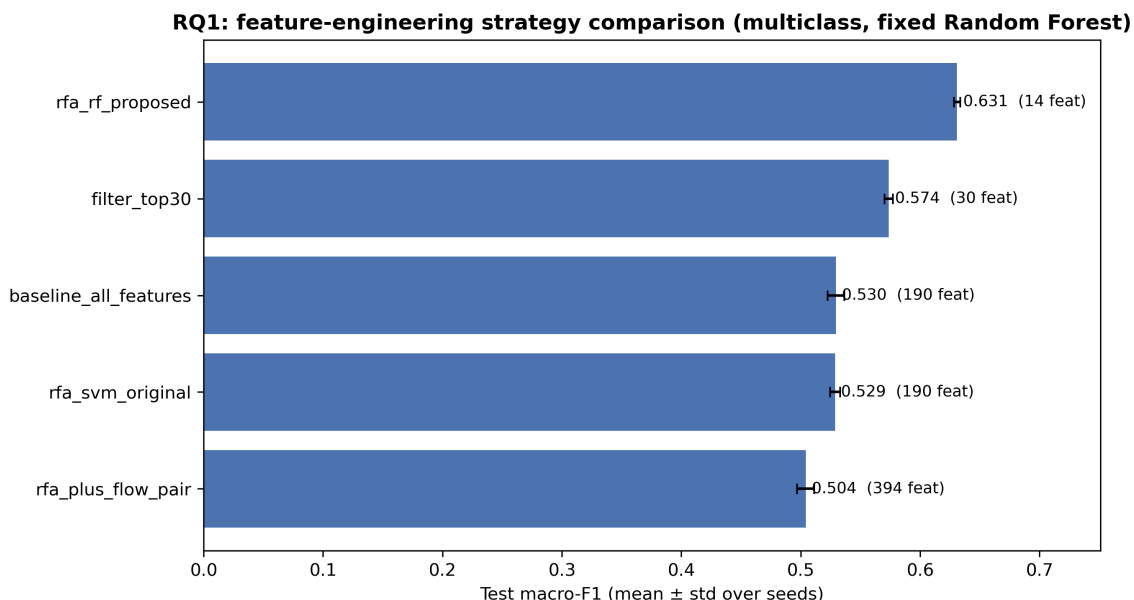


Figure 4. Feature-engineering strategy comparison (fixed Random Forest, multiclass). RFA with 14 features wins; piling on features hurts.

Why this matters. This is a real, publishable finding and a classic lesson: dimensionality is not free. The best strategy is the one that discards the most while keeping the signal.

7. Models and architectures

All six models implement the same interface and are driven by the same harness, so they are trained and scored identically. Briefly, in increasing complexity:

- **Rule-based:** a depth-5 decision tree whose paths read as human rules ("if TTL is X and bytes are Y then ..."). It is the interpretable baseline — the floor everything else must beat.
- **Random Forest:** hundreds of decision trees voting; class-balanced; fed unscaled features.
- **SVM (RBF kernel):** finds a curved separating boundary; trained on a 30,000-row stratified subsample with its own internal scaler.
- **XGBoost:** gradient-boosted trees that fix each other's mistakes in sequence; handles missing values natively; the strongest classical model here.
- **1-D CNN:** Conv1D + BatchNorm + ReLU + MaxPool blocks, then global average pooling, a dense layer, and a softmax. Convolution scans the feature vector for local patterns.
- **LSTM:** a recurrent network over short windows of consecutive records.

The deep-learning rescue (a key story). Initially the CNN and LSTM performed terribly — multiclass macro-F1 around 0.13, *below* the rule-based baseline. The cause was not the architecture but the **inputs**: the networks were being fed raw, unscaled features with values up to roughly six billion, which makes gradient-based training diverge. The fix had two parts. First, give the networks a proper input transform: a signed **log1p** on the heavy-tailed columns (to compress the enormous range) followed by z-scoring, fitted on training data only. Second, tune the optimizer: a smaller learning rate (0.0005), dropout 0.3, a learning-rate schedule that halves the rate when progress stalls, and early stopping that restores the best epoch. After these changes the deep models train cleanly and land in a sensible range (Section 8).

Why this matters. The lesson is that "deep learning doesn't work here" is often really "the inputs were wrong." Diagnosing the scaling bug, rather than abandoning the models, is what produced honest, comparable deep-learning numbers.

Check your understanding. Why are tree models unaffected by unscaled billion-valued features while neural networks are crippled by them?

8. Evaluation protocol and headline results

Every model is trained with **five random seeds** (42 to 46) and we report the mean and standard deviation, because a single run can be lucky. The metrics are chosen for an imbalanced problem:

- **Macro-F1:** the F1 score (harmonic mean of precision and recall) computed per class and then averaged with equal weight. Equal weight is the point: a rare class counts as much as a common one, so the model cannot hide poor rare-class performance.
- **Balanced accuracy:** the average per-class recall — again immune to imbalance.
- **AUROC and PR-AUC (binary):** threshold-free quality; PR-AUC is the more honest of the two under heavy imbalance.
- **Bootstrap 95% confidence intervals:** we resample results thousands of times to estimate how uncertain each number is.

- **Train vs test scoring:** we score the training fold too, so the gap between train and test exposes overfitting.

The headline test macro-F1 scores (mean over 5 seeds):

Model	Binary F1	Multiclass F1	Family
Rule-based	0.864	0.360	interpretable
SVM (RBF)	0.859	0.389	classical
Random Forest	0.919	0.534	classical
XGBoost	0.925	0.611	classical
1-D CNN (tuned)	0.905	0.434	deep
LSTM (tuned)	0.894	0.447	deep

Two clear stories. On the **binary** task (attack vs normal) everything works well; XGBoost leads at 0.925 and even the simple rule tree reaches 0.864. The **multiclass** task (naming which of ten attack types) is much harder for everyone, and here the tree ensembles clearly lead, with the deep models competitive but behind. On tabular flow features this is the expected outcome — gradient-boosted trees are simply strong on this kind of data.

The training curves below confirm the deep models are now **well-trained, not overfit**: training and validation loss descend together, with validation at or below training (a consequence of dropout). The generalization-gap chart then makes a subtler point.

Deep models: train vs. validation (mean $\pm$ std over seeds)

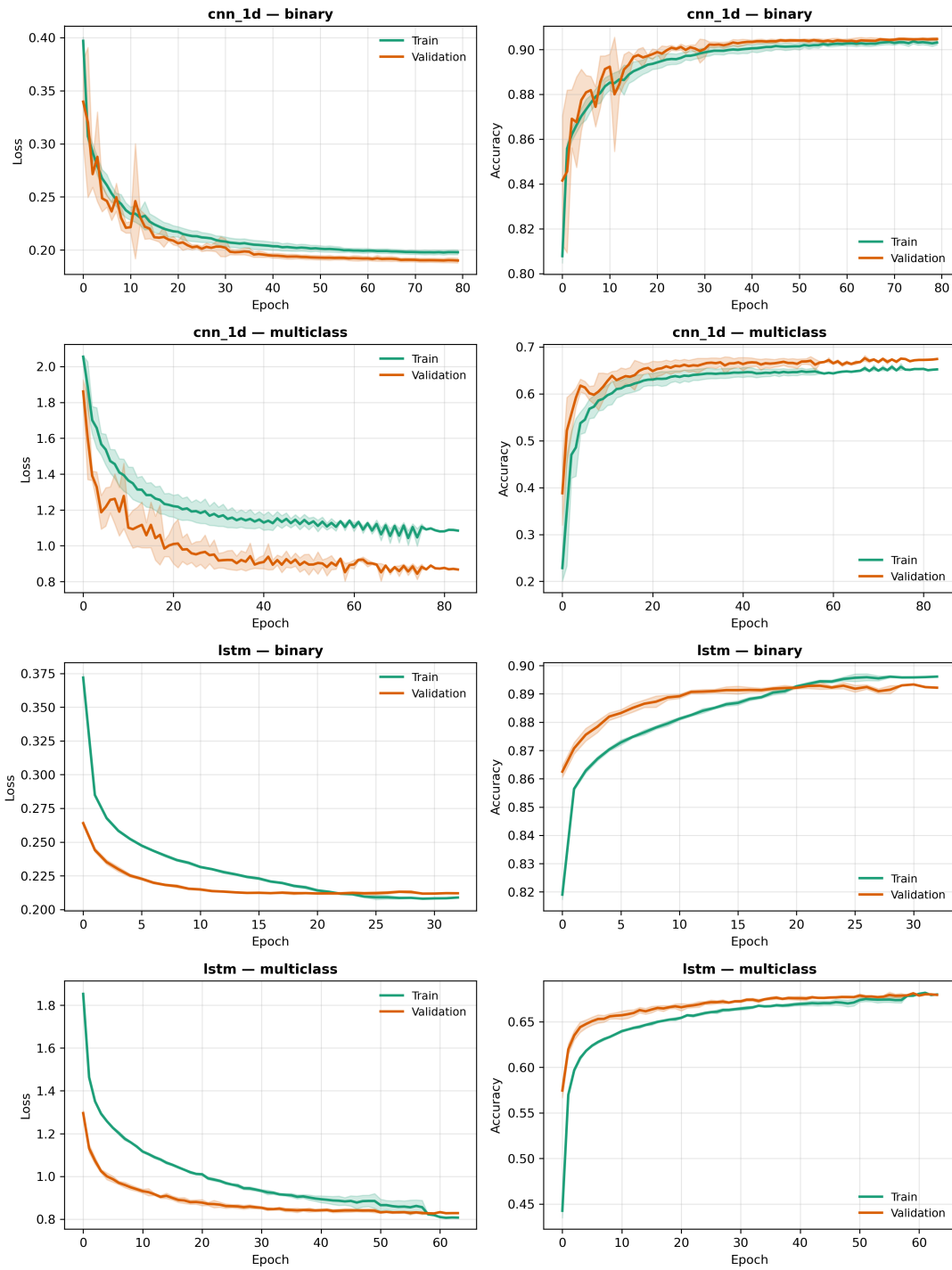


Figure 5. Deep-model train vs validation curves (mean $\pm$ std over 5 seeds). Curves track together; no diverging gap = no overfitting.

Train vs. test macro-F1 — generalisation gap by model (mean $\pm$ std over seeds)

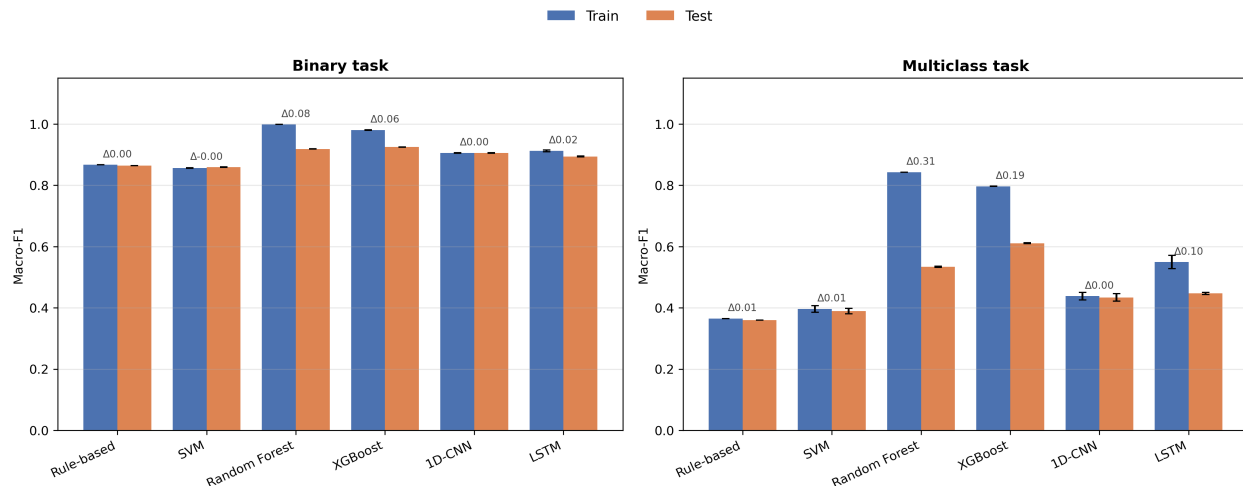


Figure 6. Train-vs-test macro-F1 gap. Tree ensembles win on test score but overfit multiclass (RF gap 0.31, XGBoost 0.19); the CNN barely overfits at all.

The gap chart reveals an honesty point the leaderboard hides: **Random Forest and XGBoost buy much of their multiclass lead through overfitting** — RF scores 0.843 on train but 0.534 on test (a 0.31 gap), XGBoost 0.797 vs 0.611. The 1-D CNN, by contrast, scores about 0.438 on both (essentially zero gap): it generalizes almost perfectly, just at a lower absolute level. This interpretability-versus-accuracy-versus-honesty trade-off is exactly the kind of nuance the RQ analysis is meant to surface.

Check your understanding. XGBoost has the best multiclass test score but a 0.19 train-test gap; the CNN scores lower but has a ~0 gap. Which would you trust more on genuinely new traffic, and why?

9. RQ1 — feature-engineering strategy and feature groups

RQ1 has two halves. The first — which *strategy* wins — was answered in Section 6: RFA with 14 features. The second half asks which *kinds* of features matter, and for which attack types. Features are grouped into five semantic families (basic flow statistics, content, timing, general-purpose, and connection-tracking), and we measure each group's importance per class by **permutation importance**: shuffle a group's columns, see how much that class's F1 drops. A big drop means the class depends on that group.

First, the winning RFA selection is dominated by **basic flow** features — 10 of its 14 features are byte counts, packet rates, TTLs and protocol/service indicators — with one feature each from the timing, content, general-purpose and connection-tracking groups. A handful of basic volume statistics carry most of the multiclass signal.

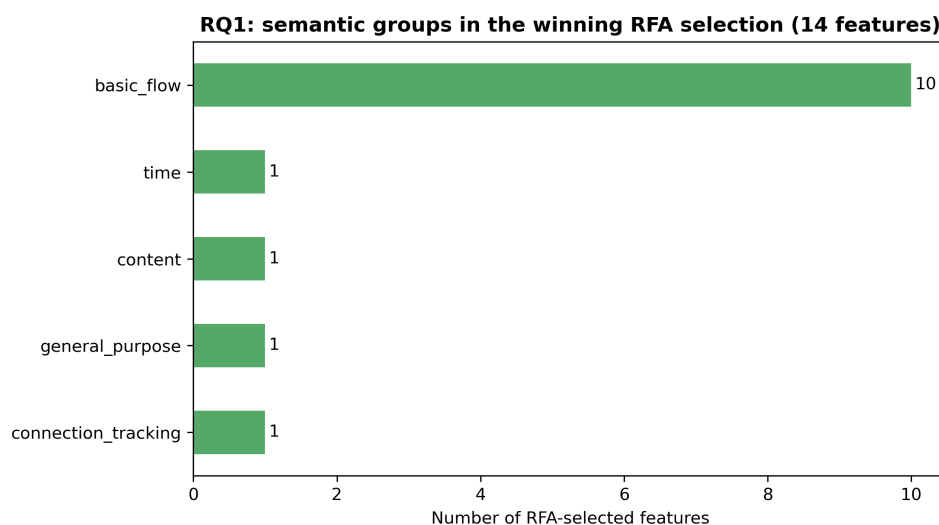


Figure 7. Semantic groups inside the 14-feature RFA selection. Basic flow statistics dominate.

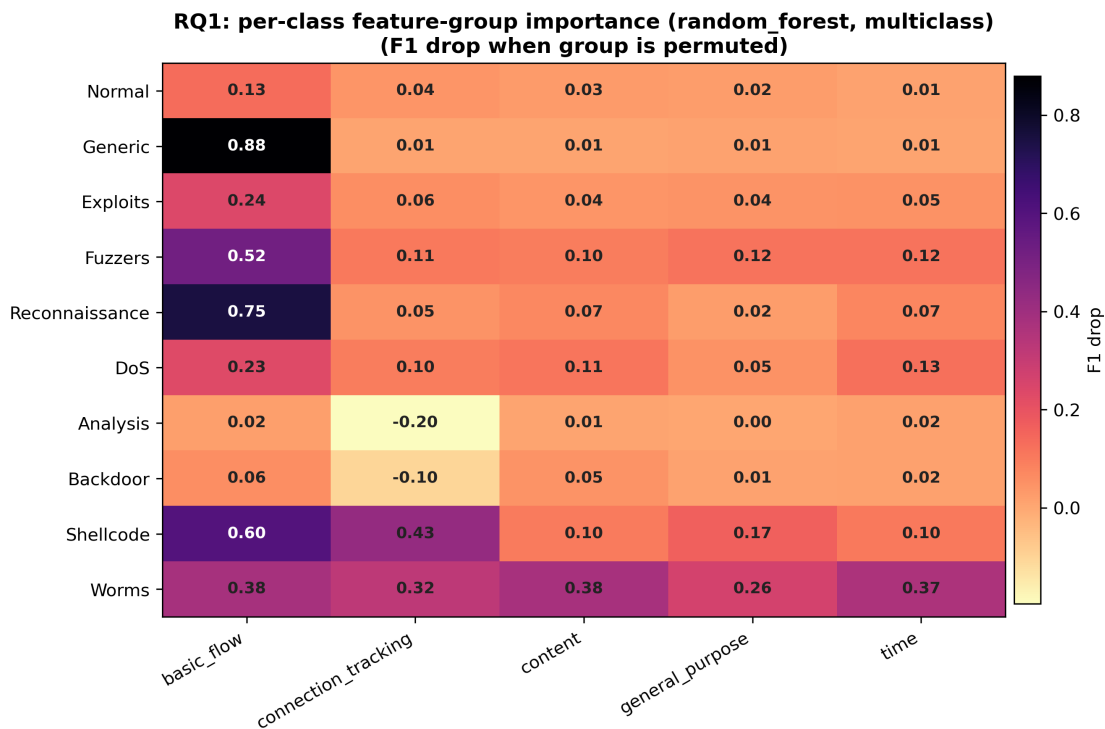


Figure 8. Per-class feature-group importance (permutation: F1 drop when a group is shuffled). Darker = more essential.

The heatmap (Figure 8) gives the per-class answer and two striking results:

- **Basic flow dominates almost everywhere.** Generic (0.88 drop) and Reconnaissance (0.75) are essentially defined by their flow-volume statistics; remove them and detection collapses.
- **Shellcode and Worms need more than flow.** Shellcode relies on basic flow (0.60) *and* connection-tracking (0.43) together; Worms is the only class with no single dominant group — its importance is spread thinly across all five (0.26 to 0.38). That diffuse signal, plus its tiny sample size, is precisely why Worms is hard.
- **A negative result that explains a confusion.** For Analysis and Backdoor the connection-tracking group has *negative* importance (-0.20 and -0.10): shuffling those features actually *improves* their F1. Those features actively cause Analysis and Backdoor to be mistaken for each other — a direct, evidence-backed bridge to RQ2.

Why this matters. RQ1 turns "which features matter" from a guess into a measured, per-class answer. The negative-importance finding is the kind of non-obvious result that makes a comparative study worth publishing.

Check your understanding. If a feature group has negative permutation importance for a class, what is that telling you about the features and that class?

10. RQ2 — which classes stay hard, and why

RQ2 asks which attack types remain hard *regardless of model*, and diagnoses the cause. We average each class's F1 across all six models to get a difficulty ranking, then read the confusion structure to explain it.

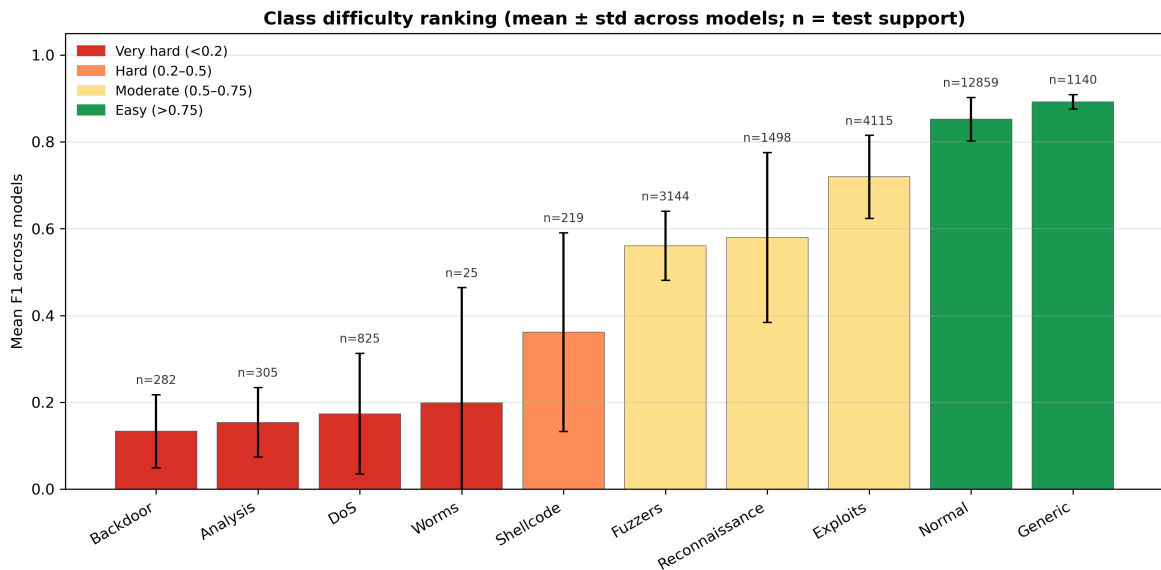


Figure 9. Class difficulty (mean F1 across all six models; n = test support). Difficulty is not simply about rarity.

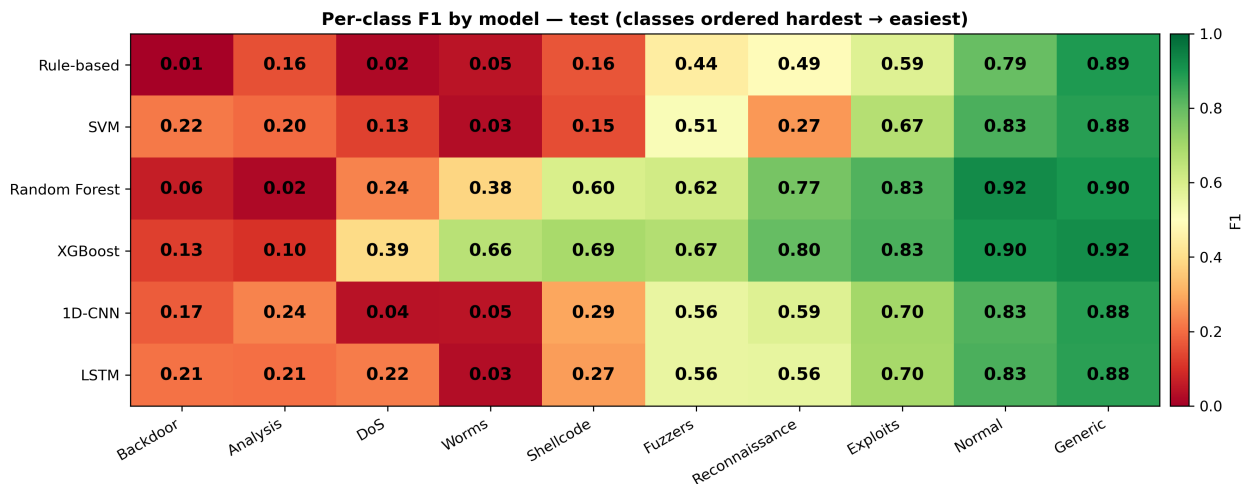


Figure 10. Per-class F1 for every model. The hard classes (left) stay red across the whole row of models.

The difficulty order, hardest first, is Backdoor (0.13), Analysis (0.15), DoS (0.17), Worms (0.20) and Shellcode (0.36); the easy end is Generic, Normal and Exploits. The heatmap shows these hard classes are red across every model column — the difficulty is intrinsic to the data, not a quirk of one model. That is the core RQ2 answer.

Why they are hard is the more interesting part, and it is not just rarity:

- **Analysis and Backdoor are mutually confused.** 49.5% of Backdoor records are predicted as Analysis and 35.1% the other way. In UNSW-NB15 these two attack types are behaviorally almost identical, and (from RQ1) their connection-tracking features make it worse.

- **Shellcode is a precision problem, not a recall problem.** Most models actually *recall* Shellcode well, but Reconnaissance records leak into it (13.6%), dragging its precision and therefore its F1 down.
- **Worms is diffuse and tiny.** Only 25 test records, frequently predicted as Exploits (25.7%), with no single feature group to anchor it — a genuinely hard case for any model.
- **DoS overlaps with Exploits** (31.1% confusion), reflecting real similarity in flow behavior.

Most-confused pairs (fraction of the true class sent to the wrong label), top five:

True class	Predicted as	Fraction
Backdoor	Analysis	49.5%
Analysis	Backdoor	35.1%
DoS	Exploits	31.1%
Worms	Exploits	25.7%
Reconnaissance	Shellcode	13.6%

Why this matters. RQ2 converts a vague worry ("some classes are hard") into specific, actionable diagnoses: which classes, how hard, and the exact confusions responsible. That is what a security team would need to decide where to invest.

Check your understanding. Shellcode has high recall but low F1. Explain, in terms of false positives, why a high-recall class can still score poorly.

11. Limitations and honest scope

Owning the project means owning its limits. The following are stated plainly and were design decisions, not oversights:

- **The LSTM's sequences are not truly temporal.** The partitioned UNSW-NB15 set does not preserve packet timestamps or flow order, so the LSTM's sliding window is over feature-space neighbors, not real time. The recurrent results should be read with that caveat; genuine temporal modeling is future work on a timestamped dataset.
- **Tree ensembles overfit on multiclass.** Their headline lead is partly memorization (Section 8), which the train-test gap chart documents rather than hides.
- **RBF-SVM is subsampled** to 30,000 rows for tractability, so its numbers reflect that subset, not the full training pool.
- **Flow-pair features are an adaptation**, cited as inspired by the original payload-bigram method, not equivalent to it.
- **Scope is Phase 1:** one dataset (UNSW-NB15), six models. Cross-dataset validation (CIC-IDS2017), anomaly detectors, and noise-robustness testing are deliberately deferred.

Check your understanding. A reviewer says "your LSTM result proves recurrent models work for IDS." Why is that claim too strong given this setup?

12. Explain this project in three minutes

"I built a fair comparison of rule-based, classical, and deep-learning intrusion detectors on UNSW-NB15, all trained through one identical, leakage-safe pipeline and evaluated over five seeds with imbalance-aware metrics. Binary attack-vs-normal detection is easy — everything scores around 0.86 to 0.93 macro-F1. Naming the specific attack type is much harder: gradient-boosted trees lead (XGBoost 0.61), the deep models are competitive (0.43 to 0.45) once I fixed an input-scaling bug that had been crippling them, and the simple baselines trail. Feature engineering matters: a 14-feature RFA selection beats using all 190 features, so less is more. The hardest classes (Backdoor, Analysis, DoS, Worms, Shellcode) are hard for every model, mostly because of mutual confusion — Backdoor and Analysis are nearly indistinguishable in this data — and I can show, via permutation importance, exactly which feature groups drive or sabotage each class. The trees win on score but overfit; the CNN generalizes better than its score suggests."

13. Glossary

SHA-256. A cryptographic fingerprint of a file; any change to the bytes changes the fingerprint.

Leakage. When information from validation/test influences training, inflating scores dishonestly.

Stratified split. A split that preserves each class's proportion in every fold.

One-hot encoding. Turning a categorical value into a set of 0/1 indicator columns.

z-score. Rescaling a feature to mean 0 and standard deviation 1.

log1p. The transform $\log(1+x)$; compresses huge values so heavy-tailed features behave.

Class weights. Telling the loss that errors on rare classes cost more.

SMOTE. Synthesizing new minority-class samples by interpolation rather than copying.

RFA. Recursive Feature Addition: greedily add the most helpful feature until it stops helping.

Permutation importance. Shuffle a feature/group and measure the score drop to gauge its value.

Macro-F1. Per-class F1 averaged with equal weight, so rare classes count fully.

Balanced accuracy. Average per-class recall; immune to class imbalance.

AUROC / PR-AUC. Threshold-free quality scores; PR-AUC is preferred under heavy imbalance.

Bootstrap CI. Uncertainty estimate from resampling results many times.

Overfitting. Learning the training data too literally; shows up as a large train-test gap.

Dropout. Randomly disabling units during training to regularize a neural network.